Tutorial: Power Trace Collection with ChipWhisperer Lite (Unmasked AES)

Logan Reichling, Tran Phuoc Dung Dang, and Boyang Wang

Data and System Security Lab (DaSec), ECE Department, University of Cincinnati
Last modified: August 2026, Status: Completed

1 Overview

This document provides and explains the setup and workflow required in order to collect power traces of an unmasked AES software implementation using ChipWhisperer Lite, where the power traces can be utilized to perform side-channel analysis. An *unmasked* implementation means that the implementation is correct and functional, but does not include additional countermeasure in the code to mitigate side-channel leakage.

This document is primarily based on information from ChipWhisperer website as well as our lab's internal tutorial document from (Logan Reichling, a PhD student in DaSec Lab), which covers the basic setup of ChipWhisperer Level-1 Kit with unmasked AES software implementation. Our jupyter notebook used in this document is based on the one from ChipWhisperer repository but with more customized code blocks for running additional side-channel analysis.

Acknowledgments. The research and education efforts associated with this document are supported by CHEST, the NSF IUCRC Center for Hardware and Embedded Systems Security and Trust.

2 Software and Hardware Settings

In this document, we use the following hardware and software.

- Hardware: ChipWhisperer Lite with an ARM STM32F3 micrcontroller as a target, USB cable (both available from CW Lite).
- Operating System: Ubuntu 22.04 (or a VM with Ubuntu 22.04), several Linux packages, Python packages, and our customized jupyter notebook on GitHub repository [1]

3 ChipWhisperer Environment Setup

3.1 ChipWhisperer Overview

ChipWhisperer is an open source toolchain dedicated to hardware security research. This toolchain consists of several layers of open source components, including hardware, firmware, and

[1] https://github.com/UCdasec/Tutorial-CW

software. More information regarding ChipWhisperer can be found on ChipWhisperer's GitHub repository[2] and official website[3].

There are multiple ChipWhisperer hardware available on the market. In this tutorial document, we will use a ChipWhisperer Lite (with an ARM STM32F3 microcontroller as a target). An picture of a CW Lite (with an AVG XMEGA microcontroller as a target) from ChipWhisperer website[4] is presented in Fig. 1. The CW main board (on the left side of the picture) is in charge of communicating with the target and capturing power traces from the target board. The target board (on the right side of the picture) is where we can upload a binary (e.g., a binary of AES encryption program) and execute it on the target microcontroller.

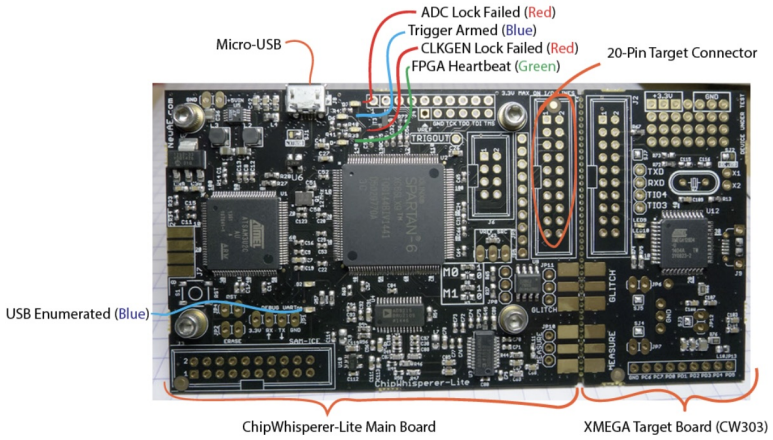

Fig. 1: CW Lite with an AVR XMEGA microcontroller as a target

ChipWhisperer Lite (with an ARM STM32F3 microcontroller as a target) shares the same main board and same layout of the target board but with an ARM STM32F3 microcontroller as a target. A picture of it can be found below.

3.2 ChipWhisperer Installation

Before we connect CW Lite hardware to our computer, we first need to install the ChipWhisperer environment. Specifically, we can follow the instructions and steps from this ChipWhisperer webpage[5] to install the ChipWhisperper environment. We assume that we are using a computer running Ubuntu or a VM running Ubuntu. We can either choose (1) *the quick installation option* by running all the commands at once or *the manual installation option* by running these commands step by step mentioned on the webpage. We recommend the manual installation option. Specifically,

[2] https://github.com/newaetech/chipwhisperer
[3] https://chipwhisperer.readthedocs.io/en/latest/getting-started.html
[4] https://chipwhisperer.readthedocs.io/en/latest/getting-started.html
[5] https://chipwhisperer.readthedocs.io/en/latest/linux-install.html

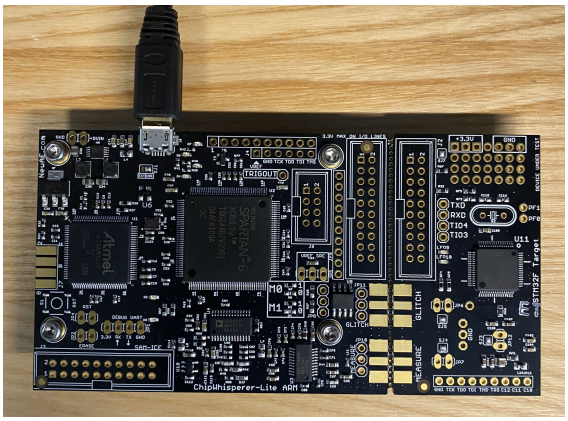

Fig. 2: CW Lite with an ARM STM32F3 microcontroller as a target

we can start from the block below and continue all the remaining steps until we reach the end of the webpage. The entire process takes **8∼15 minutes** depending on the system. We also provide a short video[6] walking through each step during the manual installation.

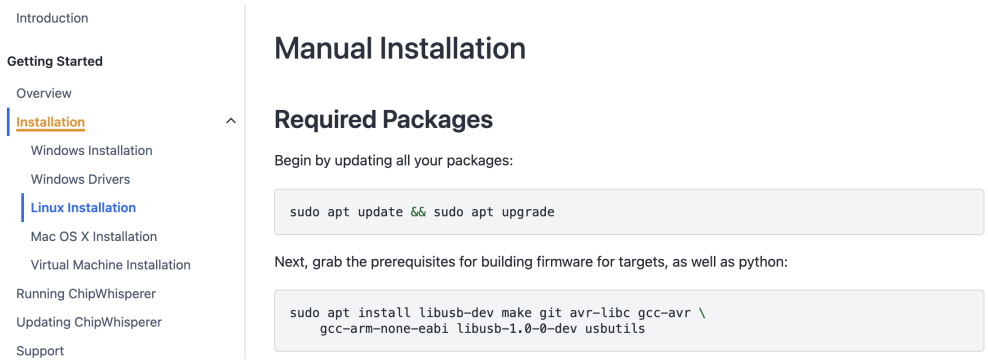

Fig. 3: Manual installation for ChipWhisperer (from ChipWhisperer website)

Make sure we reboot the system as instructed during the process. At the end of the installation, we should be able to enable the ChipWhisperer environment and run juypter notebook successfully. Specifically, when we run

```
source ~/.cwvenv/bin/activate
```

We will see the "**(.cwvenv)**" to the left of working directory if the environment is installed successfully

```
dung-dang@dungdang-ThinkPad-T15p-Gen-3:~$ source ~/.cwvenv/bin/activate
(.cwvenv) dung-dang@dungdang-ThinkPad-T15p-Gen-3:~$
```

[6] https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Installation/1_zytbs228

Next, we can open the jupyter lab by running the command below in the terminal

```
jupyter lab
```

The Jupyter server will begin to boot. Wait until the Jupyter lab automatically opens in the web browser and we should be able to see similar as shown below.

When we start Jupyter Lab, it runs as a foreground process and takes over that terminal window to keep the server alive. This blocks the current terminal session, preventing us from executing additional commands in that specific window. If we need to run other commands, we can open a new terminal tab or press **"Ctrl' + "C"** then press **"Y"** to exit the Jupyter lab.

3.3 Connect and Verify Hardware

Now, we can connect ChipWhisperer Lite to our computer by using the USB cable (offered by CW Lite box). The computer should be able to automatically recognize the CW Lite. If the connection is successful, we should observe the blue light on the board is ON and flashes (as shown in Fig. 7).

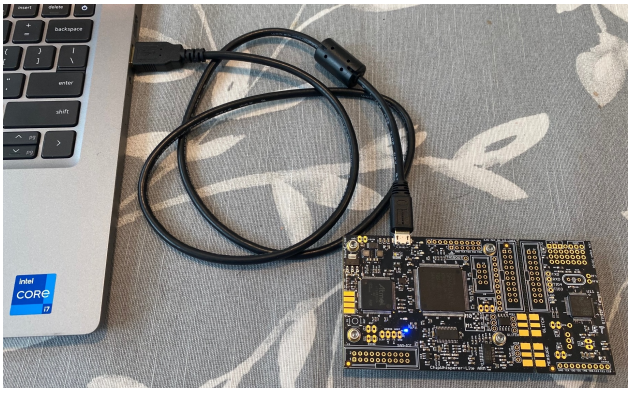

Fig. 4: CW Lite is connected to a computer (blue light flashes)

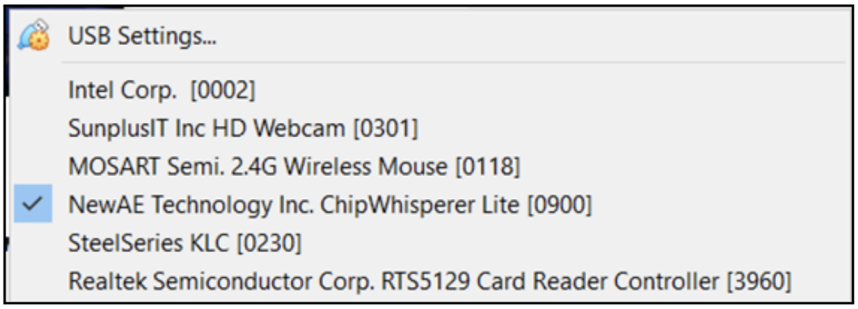

Fig. 5: Dropdown menu of VM

If we use a VM, make sure we attach the ChipWhisperer USB to the VM through the USB dropdown (as shown in Fig. 5)

Next, run the following command to enable ChipWhisperer environment

```
source ~/.cwvenv/bin/activate
jupyter lab
```

In Jupyter notebook window, click and choose `chipwhisperer` ⇒ `jupyer` ⇒ `Setup_Scripts`, and open `Setup_Generic.ipynb`.

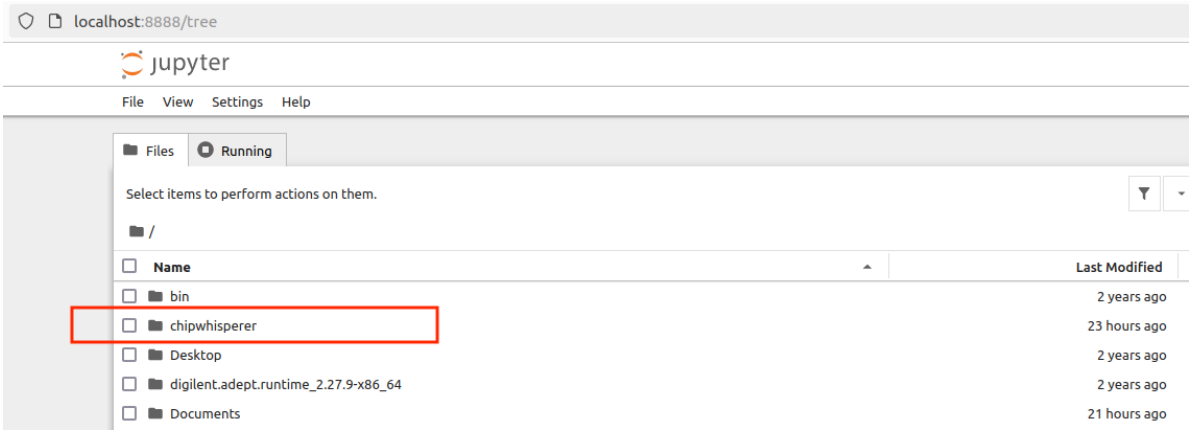

Fig. 6: Jupyter notebook (`./chipwhisperer`)

Execute the first code block of `Setup_Generic.ipynb` by clicking "Run" as shown in Fig. 8. After 1 or 2 seconds, we should observe it outputs `INFO: Found ChipWhisperer`, which indicates our computer recognize the CW Lite hardware successfully. At the same time, we should also observe that an additional green light is ON and flashes on the board. We also provide a short video[7] walking through each step during this hardware connection testing process.

[7] `https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Connection_Verify/1_800go4yr`

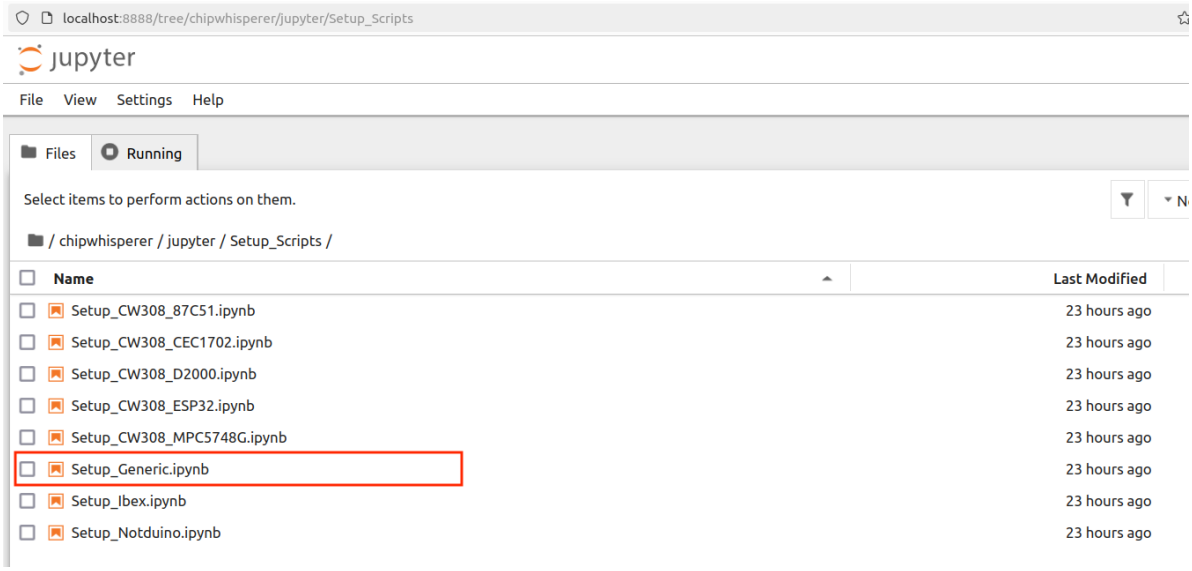

Fig. 7: Jupyter notebook (`./chipwhisperer/jupyter/Setup_Scripts`)

Fig. 8: Run the first code block of `Setup_Generic.ipynb`

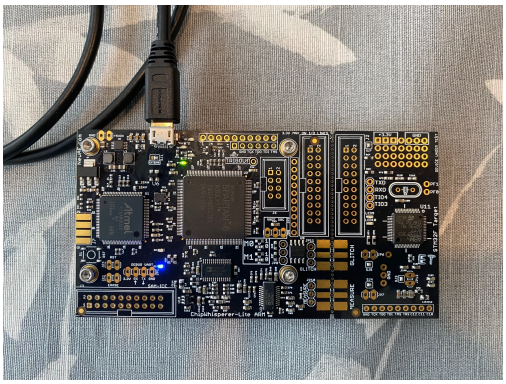

Fig. 9: CW Lite is recognized by the computer (green light flashes)

4 Power Trace Collection of Unmasked AES

4.1 Install Additional Libraries and Clone Our Repository

Assuming we have installed ChipWhisperer environment, connect and verify the connection to CW Lite hardware successfully as described above, we now install a few additional libraries needed for our customized jupyter notebook and download it.

- Close and exit the previous jupyter sessions (Ctrl + C)
- Ensure that CW Lite hardware is connected
- Enable the CW environment is enabled by running the following command again

```
source ~/.cwvenv/bin/activate
```

- Install the following Python packages with the following command. These packages are needed for our customized jupyter notebook.

```
pip install ipython scikit-learn PyCryptodomex tqdm reportlab h5py
```

- Ensure we are working in the home directory by typing

```
cd ~/
```

- Clone our GitHub repository[8] by running the following command

```
git clone https://github.com/UCdasec/Tutorial-CW.git
```

4.2 Unmasked AES Implementation

We leverage an unmasked AES implementation, named as TinyAES, which is supported by ChipWhisperer. A copy of this implementation can be found here[9]. The implementation includes two files: `aes.c` and `aes.h`. By default, the program will run AES-ECB-128 encryption with one plaintext block, where the key has 128 bits and the plaintext also has 128 bits. The output is a ciphertext with 128 bits. The ECB mode is deterministic, which is not secure to use in practice, even without considering side-channel analysis. However, since there is one plaintext block, it is easier to demonstrate side-channel analysis with ECB encryption without the involvement of Initialization Vectors (i.e., randoms) from other modes. Side-channel analysis on other modes can be easily expanded from ECB by adding one additional step to remove the IVs with an XOR operation. That is why most of the side-channel analysis focuses on power/EM traces from the ECB mode of AES only.

We can also modify or customized the C code of this TinyAES128 if needed. In this tutorial, we use the default one from ChipWhisperer without any modifications.

[8] https://github.com/UCdasec/Tutorial-CW
[9] `https://github.com/newaetech/chipwhisperer/tree/develop/firmware/mcu/crypto/tiny-AES128-C`

4.3 Run the Collection

To run the power trace collection of this TinyAES on ChipWhisperer Lite, we can follow the steps below. We also provide a short video[10] walking through each step for the trace collection as well as side-channel analysis with TVLA and CPA.

1. Open Terminal and run the command below to enable the environment to enable the Jupter Notebook if needed

```
source ~/.cwvenv/bin/activate
jupyter lab
```

   A web browser tab for jupyter notebook will open.
2. Connect ChipWhisperer Lite to our computer and ensure only one blue light is ON and flashes (if not, gently disconnect the USB cable and reconnect). Otherwise, we may get `scope() not defined` error later on.
3. In the web browser for Jupyter notebook, direct to "/Tutorial-CW/Setup_Scripts", then click on the "**Setup_CW_Lite_TinyAES.ipynb**". This script contains multiple code cells to initialize the settings, compile and load a binary to the target board, and collect and save power traces while the binary is running on the target board. We will explain each cell one by one below.
4. Before we run any cells in this notebook, we can restart the kernel and clean all the outputs from previous runs.

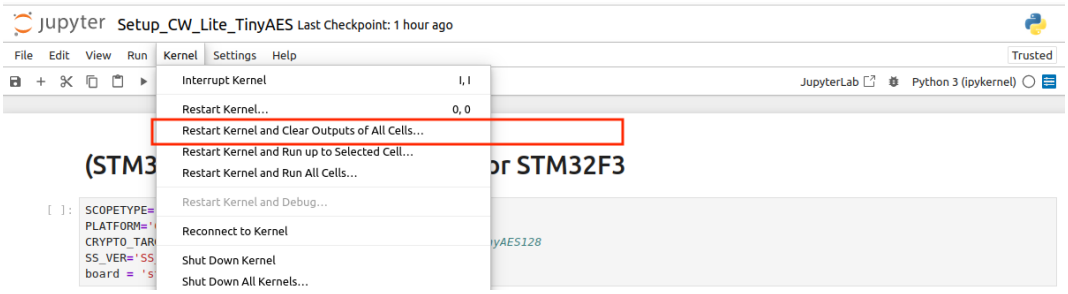

Fig. 10: Restart kernel and clean all the outputs

5. For the first two cells, we only need to select one cell to run depending on the target board we choose (either STM32F or XMEGA). We use STM32F only in this document so we will choose to run the STM32F one. Make sure the `SCOPETYPE` is `OPENADC`, `PLATFORM` is `CW308_STM32F3` and `CRYPTO_TARGET` is `TINYAES128C`. *To run an individual cell, select the cell by left-clicking on it, then press the "Run" button*

[10] `https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Lite_Unmaksed_AES/1_fpzm0kla`

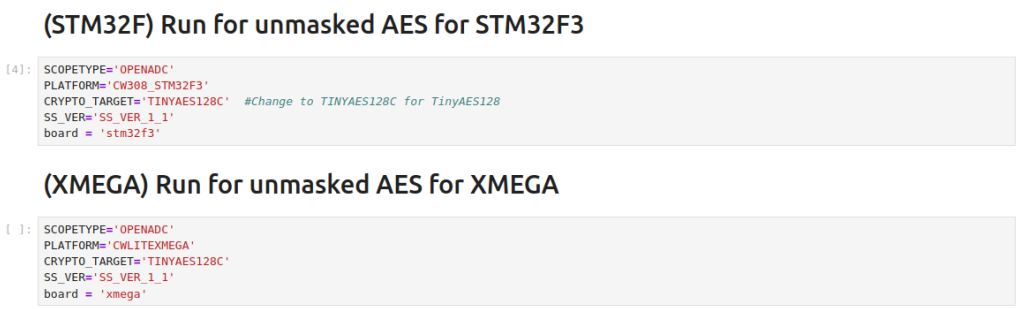

Fig. 11: Choose the target board setting (Run the STM32F one only)

6. Next, we run "Set Offset and Samples per Trace Parameter" cell. `scope.adc.offset` indicates when ChipWhisperer should start collecting samples in each trace (0 by default). `scope.adc.samples` suggests how many samples we would like to collect per trace (5,000 by default, CW Lite can support up to 5,000 per trace).

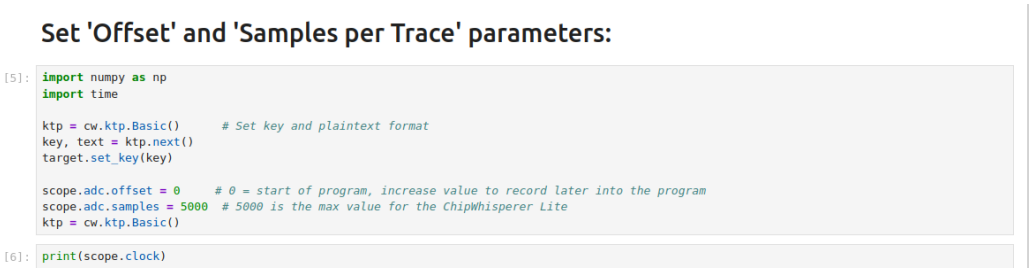

Fig. 12: Choose offset and samples per trace

After run the two cells, we should observe similar information below where the target operates at 7.38MHz and the sampling rate of our collection is 29.53MHz (4 times of the target clock).

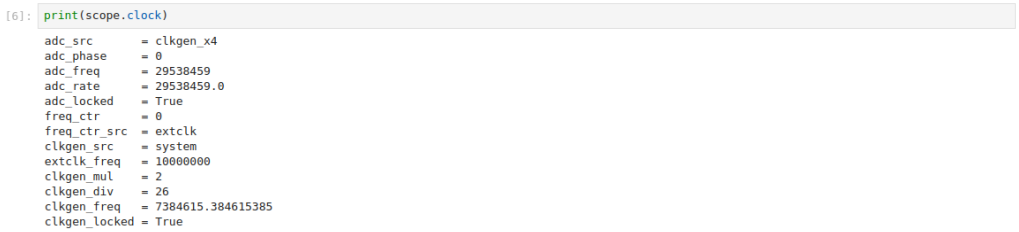

Fig. 13: Information from `scope.clock()`

7. Next, we set up the encryption key that we will use in this trace collection by running "Key Setup" cell. ChipWhisperer can generate a random key for us by default. In addition, we can also choose a specific key by updating `key_str`.

Fig. 14: Choose the encryption key for trace collection

We should see the key printed out. In addition, we can also run the next cell to check a small number of plaintexts randomly generated by ChipWhisperer but the key remains the same.

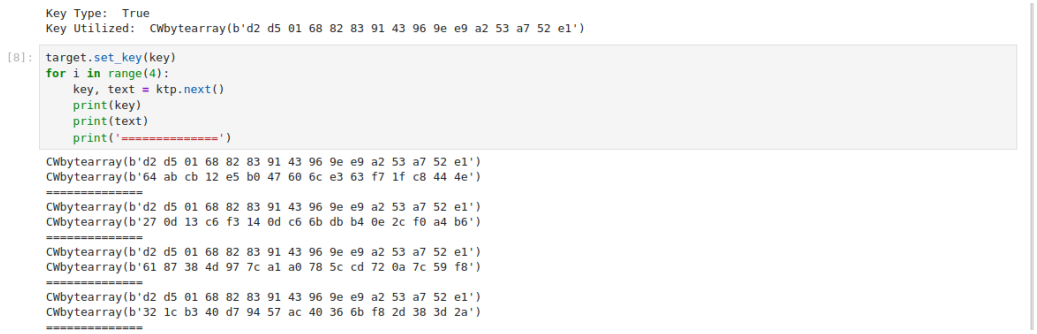

Fig. 15: Test plaintexts are random and different

8. We can run the **first cell** from "Detect, Compile, and Flash to ChipWhisperer" to detect the CW Lite hardware. If successful, we should see the message `INFO: Found ChipWhisperer` again along with some scope information.

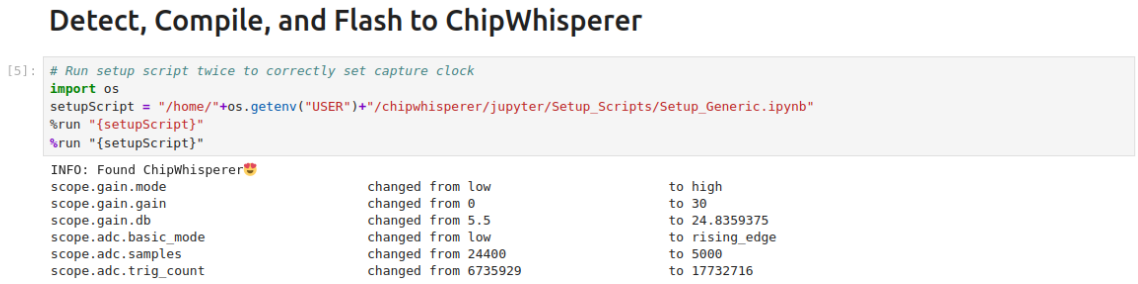

Fig. 16: Detect CW Lite

9. Next, we run the **second cell** from "Detect, Compile, and Flash to ChipWhisperer" to compile the C code of TinyAES. We should observe binary (`simpleserial-aes-CW308_STM32F3.elf`) is generated successfully.

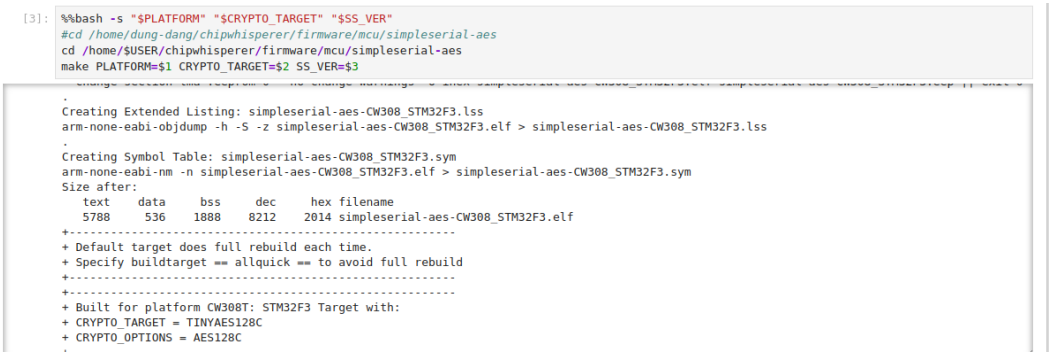

Fig. 17: Compile the C code of TinyAES

10. Next, we run the **third cell** from "Detect, Compile, and Flash to ChipWhisperer" to flash the binary to the target board. We should see the message saying "Verified flask OK" for the binary file in hex format.

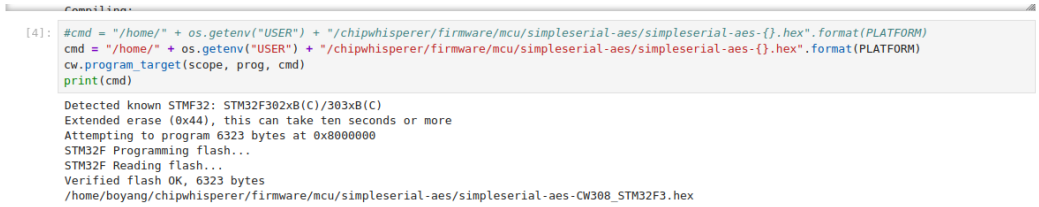

Fig. 18: Flash the binary to target board

11. Next, we collect a small number of power traces, e.g., 10 or 20, to see if the collection is able to operate with multiple iterations by running "Collect A Small Number of Sample Traces for Initial Testing."

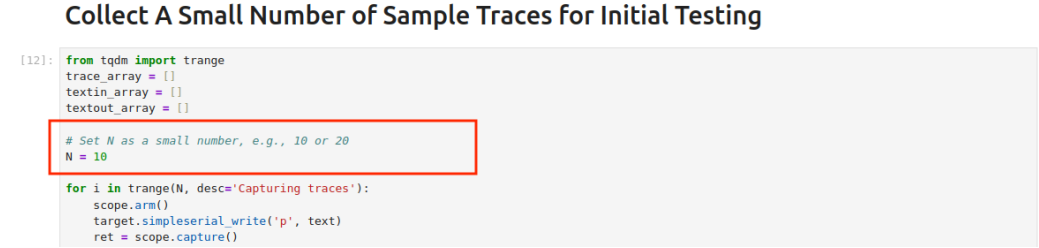

Fig. 19: Collect a small number of N power traces for initial testing

12. We can visualize the shape of a power trace we collect by running "Visualize the Shape of a Collected Power Trace"

Visualize the Shape of A Collected Power Trace:

```
[24]: %matplotlib inline
import matplotlib.pylab as plt
from matplotlib.pyplot import MultipleLocator

def annotate_zone(ax, start, end, text, color="red", h_offset=0.0):
    ymin, ymax = ax.get_ylim()
    y_arrow = ymin + (ymax - ymin) * (0.75 + h_offset)

    ax.axvspan(start, end, color=color, alpha=0.15)
    ax.annotate('', xy=(start, y_arrow), xytext=(end, y_arrow), arrowprops=dict(arrowstyle="<->", color=color, lw=2))
    ax.text((start + end)/2, y_arrow + (ymax-ymin)*0.02, text, color=color, ha="center", va="bottom", fontweight="bold")

offset = 0
startPOI = 0
endPOI = 5000

plt.figure(figsize=(13,8))
ax = plt.gca()
ax.xaxis.set_major_locator(MultipleLocator(1000))
```

Fig. 20: Visualize the shape of a power trace

We can observe a plot similar as below (Note: the shape of the trace varies across different binaries, implementations, optimization levels, systems, compilers, etc.)

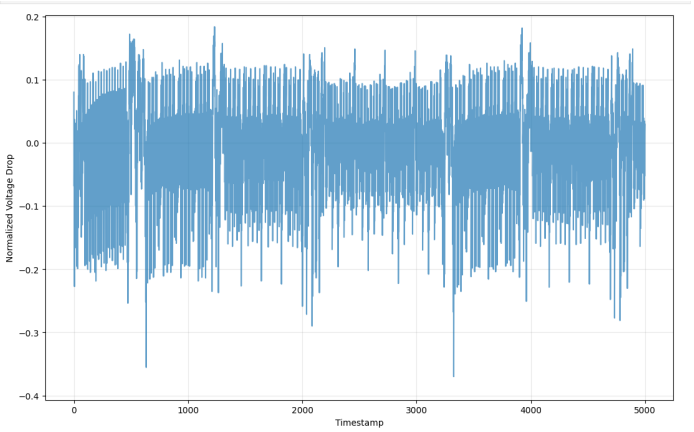

Fig. 21: An example trace of TinyAES on CW Lite (Arm STM32F3)

Note: With 5,000 samples, it actually only reaches the start of MixColumn from 2nd round of AES. In other words, it does not contain all the samples of the entire AES encryption for one block. However, since we will use samples of SubBytes operation from the 1st round of AES, these 5,000 samples are sufficient. SubBytes of the 1st round start around 1,200 timestamp and end around 2,100 timestamp. Locating the Points of Interest (POIs) associated with SubBytes (or any function in the C code) requires some additional reverse engineering process (e.g., adding 20∼50 NOP operations at the start of a function to create a visible gap in power trace). We

skip the details here. It is also easy to observe there are repeat patterns from functions, such as AddKeys and SubBytes, from 1st round and 2nd round in this example trace.

13. Next, by running "Verify Ciphertexts Were Calculated correctly" cell, we verify the encryption function can generate correct ciphertexts by comparing the outputs with another crypto library (`PyCryptodomex`). This verification is critical, especially when we need to modify the source code of AES.

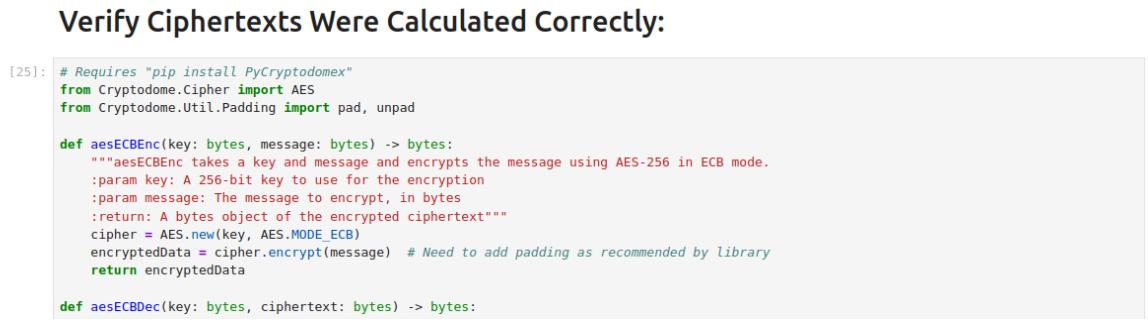

Fig. 22: Verify the ciphertexts

If successful, we can observe similar information below

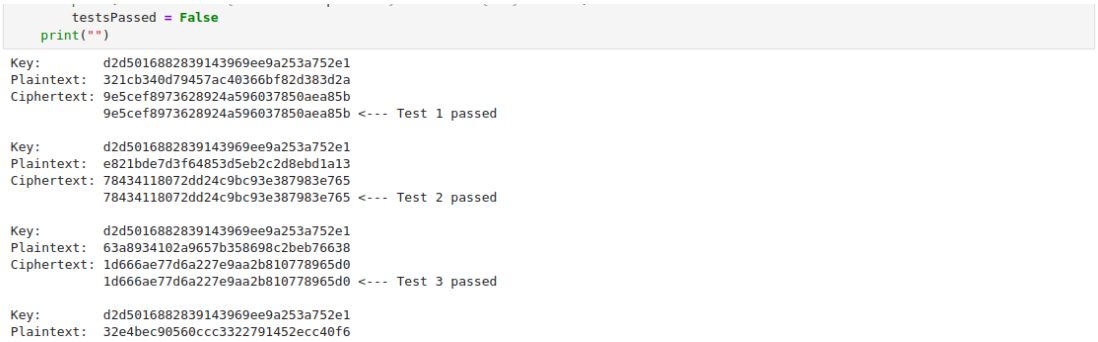

Fig. 23: Output from ciphertext verification

14. If everything looks promising so far, we can now collect power traces in a large scale by running "Collect N Power Traces for Side-Channel Analysis" cell. Modify N the number of power traces we would like to collect before running the cell. For this TinyAES, choose $N = 5,000$, which should be sufficient to recover keys later on. It takes about 3∼4 minutes to collect 5,000 traces on this target (depending on the system).

Collect N Power Traces for Side-Channel Analysis

```
[26]: import time
import warnings
import matplotlib.pyplot as plt
warnings.filterwarnings("ignore", category=UserWarning, module="matplotlib.*")
trace_array = []
textin_array = []
collectionStart = time.time()

# N is the number of traces we would like to collect,
# For TinyAES (i.e., unmasked AES), typically 5,000 traces would be sufficient to recover key bytes.
N = 5000

for i in trange(N, desc='Capturing traces'):
    scope.arm()
```

Fig. 24: Collect N power traces in a large scale

15. Next, we can save the *power traces* we collected, along with the *plaintexts* and the *key* into a .npz file by running "Save Collected Traces" cell. We can customize the path if needed.

Save Collected Traces:

```
In [ ]: from datetime import datetime
# Save traces under the following filename, naming should follow these conventions:
# <Device Type Indicator Letter><Device Number>_K<Key Number>_<Number of traces in thousands>.npz
datasetName='STM32_K2_5k_TinyAES_20260802-001'
datasetExtension = ".npz"
output_path="/home/boyang/Documents/"+datasetName  # Save to Ubuntu Desktop
os.makedirs(output_path, exist_ok=True)

outpath = os.path.join(output_path, datasetName+datasetExtension)
np.savez(outpath, power_trace=trace_array, plain_text=textin_array, key=key)  # NPZ Format
```

Fig. 25: Save N power traces, N plaintexts, and the key into a .npz file

16. At the end, we can also run the "Export Metadata" cell to save the metadata by generating an additional log file and a dataset info file for better reproducibility.

Export Metadata:

```
[25]: # Export collection metadata to a text document
import hashlib
exportLogSavePath = os.path.join(output_path, "reproducibilityLog.log")
exportLog = list()
exportLog.append(f"################################################################################")
exportLog.append(f"### Collection finished at {datetime.today().strftime('%m_%d_%Y %H:%M:%S')} by {os.getenv('USER')}")
exportLog.append(f"### Collection process took {collectionDuration:.2f}s.")
exportLog.append(f"################################################################################")
exportLog.append(f"Target Parameters: ")
exportLog.append(f" * Target Board: {board}")
exportLog.append(f" * Target Implementation: {CRYPTO_TARGET}")
exportLog.append(f" * Target Platform: {PLATFORM}")
exportLog.append(f" * Simpleserial Version: {SS_VER}")
exportLog.append(f" * Target programmer: {prog}")
```

Fig. 26: Export metadata for better reproducibility

5 Run Side-Channel Analysis on a Dataset

Within our juypter notebook `Setup_CW_Lite_TinyAES.ipynb`, we can also run the side-channel analysis based on the dataset we just collected or a dataset previously collected by others. Specifically, we offer two side-channel analysis methods, TVLA and CPA, in our juypter notebook.

5.1 Run Side-Channel Analysis with Power Traces

Assuming we just completed our power trace collection above and have not closed the jupyter notebook

TVLA. To run TVLA on the power traces we just collected, first run "TVLA Functions" cell,

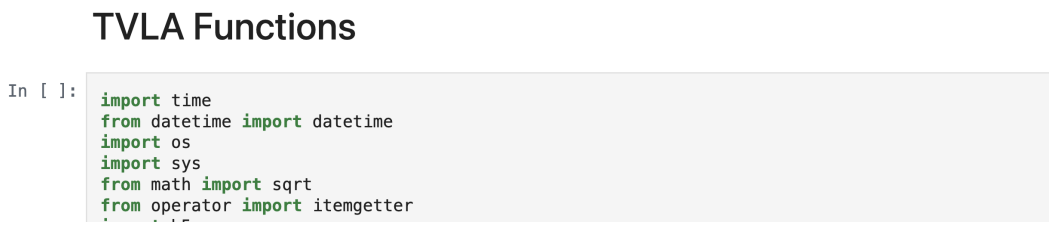

Fig. 27: Run TVLA functions cell

Then, run "Run TVLA and CPA on Collected Traces" cell. Before we run it, we can enable `plot3Activities` function (for TVLA) but disable `plot4Activities` (for CPA). We can choose which target key byte we would like to analyze by updating `target_byte` (a value from 0 to 15 as the key has 16 bytes). A TVLA plot will also be saved to the path we provide (in this case, `home/user/Documents/TVLA-Graph1.jpg`). We can change the path if needed. Recovering the entire key requires running the analysis 16 times (one for each key byte).

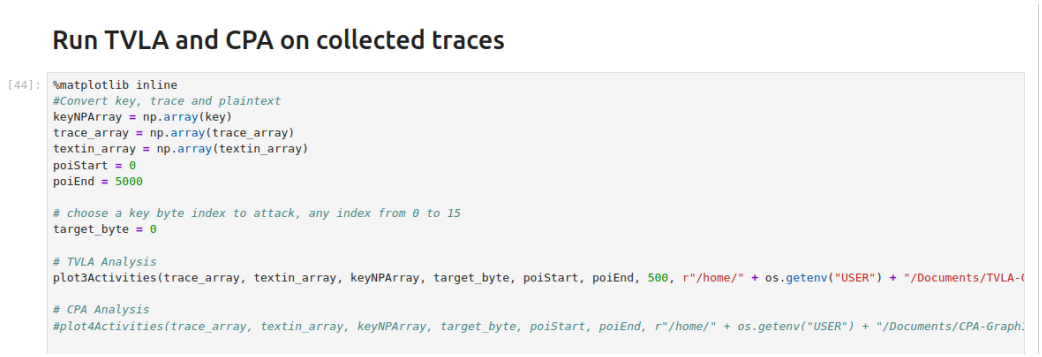

Fig. 28: Run TVLA

The analysis may take a few minutes or longer depending on the number of traces as well as the number POIs (Points of Interest) selected (`poiEnd - poiStart`). If the attack is successful,

we should observe similar as below, where the highest peak of TVLA values beyond 4.5 and the timestamp(s) are associated with the corresponding byte of SubBytes operation (1st round of AES). Other high peaks are typically associated with key loading as well as other functions. ***Note:*** *TVLA only indicates that a high side-channel leakage potentially exists, but does not reveal keys directly.*

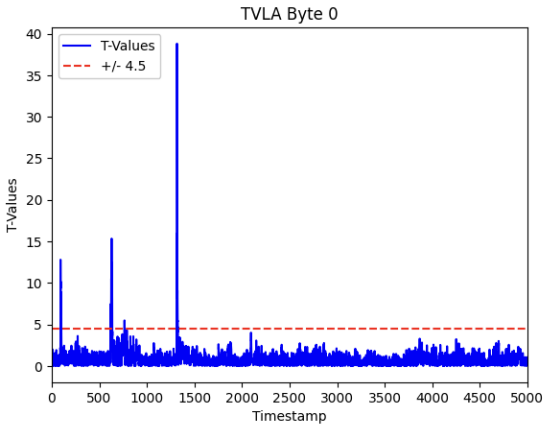

Fig. 29: Results from TVLA on one key byte

CPA. To run CPA on the power traces we just collected, run "CPA Functions" cell first

CPA Functions

In [ ]:

```
import time
from datetime import datetime
import numpy as np
from tqdm import tqdm
import matplotlib.pyplot as plt
import sklearn
```

Fig. 30: Run CPA functions cell

Then, run "Run TVLA and CPA on Collected Traces" cell. Before we run it, we can enable `plot4Activities` function (for CPA) but disable `plot3Activities` (for TVLA). We can choose which target key byte we would like to attack by updating `target_byte` (a value from 0 to 15 as the key has 16 bytes). A CPA plot will also be saved to the path we provide (in this case, `home/user/Documents/CPA-Graph1.jpg`). We can change the path if needed. Recovering the entire key requires running the analysis 16 times (one for each key byte).

Run TVLA and CPA on collected traces

```
[44]: %matplotlib inline
#Convert key, trace and plaintext
keyNPArray = np.array(key)
trace_array = np.array(trace_array)
textin_array = np.array(textin_array)
poiStart = 0
poiEnd = 5000

# choose a key byte index to attack, any index from 0 to 15
target_byte = 0

# TVLA Analysis
# plot3Activities(trace_array, textin_array, keyNPArray, target_byte, poiStart, poiEnd, 500, r"/home/" + os.getenv("USER") + "/Documents/TVLA

# CPA Analysis
plot4Activities(trace_array, textin_array, keyNPArray, target_byte, poiStart, poiEnd, r"/home/" + os.getenv("USER") + "/Documents/CPA-Graph1
```

Fig. 31: Run CPA and show results

The analysis may take a few minutes or longer depending on the number of traces as well as the number POIs (Points of Interest) selected (`poiEnd - poiStart`). If the attack is successful, we should observe similar as below, where the correct key guess has the highest correlation value (i.e., distinguishable compared to other key guesses) and is the same as the actual key byte (210 in decimal or `D2` in hex in this example plot). In case CPA cannot recover the key byte, no key guess will have a correlation value that is distinguishable compared to others.

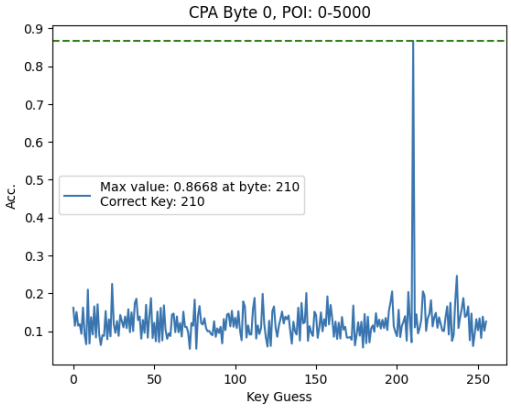

Fig. 32: Results from CPA on one key byte

5.2 Run Side-Channel Analysis with A Previous Dataset

We first load a dataset collected previously by running "Read the data from a .npz file" cell. Make sure we update the path correspondingly to the path of the dataset.

Read the data from a .npz file

```
[22]: import numpy as np
data = np.load("/home/boyang/Documents/STM32_K2_5k_TinyAES_20260802/STM32_K2_5k_TinyAES_20260802.npz")
trace_array = data['power_trace']
textin_array = data['plain_text']
key = data['key']

#print(data['key'])
print(key)
print(trace_array)
print(textin_array)
print(trace_array.shape)
print(textin_array.shape)

data.close()
```

Fig. 33: Load a dataset

If successful, we should observe the information of the loaded dataset.

```
[210 213   1 104 130 131 145  67 150 158 233 162  83 167  82 225]
[[ 0.08105469 -0.06445312 -0.03613281 ... -0.04882812  0.01757812
   0.03320312]
 [ 0.08398438 -0.06445312 -0.03710938 ... -0.0546875   0.01367188
   0.02734375]
 [ 0.07910156 -0.06542969 -0.0390625  ... -0.05859375  0.00976562
   0.02636719]
 ...
 [ 0.08203125 -0.06933594 -0.0390625  ... -0.04980469  0.01855469
   0.03027344]
 [ 0.08496094 -0.06640625 -0.04003906 ... -0.05273438  0.015625
   0.03027344]
 [ 0.08007812 -0.07128906 -0.04101562 ... -0.05664062  0.01269531
   0.02832031]]
[[  3  57 225 ... 161  63 254]
 [128 171  39 ...  88 175 194]
 [ 62 229 121 ... 112 170 150]
 ...
 [ 37 123 203 ... 136 206  85]
 [163  77  40 ... 251 218  29]
 [186 182  41 ... 153 153  17]]
(5000, 5000)
(5000, 16)
```

Fig. 34: Load a dataset

Next, we can run the steps covered in the previous subsection to run TVLA and CPA on this loaded dataset.

6 Conclusion

This concludes this tutorial.